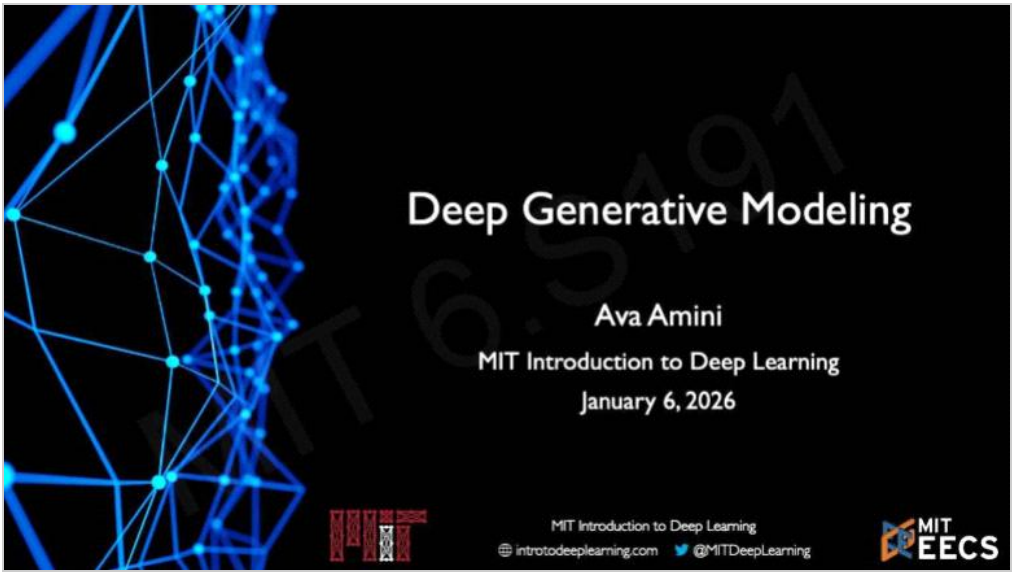
Lecture 4

Wednesday, 22 April 2026 10:36 am

6S191_MIT
_DeepLea...



Supervised vs unsupervised learning

Supervised Learning

Data: (x, y)
x is data, y is label

Goal: Learn function to map
$x \rightarrow y$

Examples: Classification, regression, object detection, semantic segmentation, etc.

Unsupervised Learning

Data: x
x is data, no labels!

Goal: Learn some *hidden* or *underlying structure* of the data

Examples: Clustering, feature or dimensionality reduction, etc.

Supervised vs unsupervised learning

Supervised Learning

Data: (x, y)
x is data, y is label

Goal: Learn function to map
$x \rightarrow y$

Examples: Classification, regression, object detection, semantic segmentation, etc.

Unsupervised Learning

Data: x
x is data, no labels!

Goal: Learn the *hidden* or *underlying structure* of the data

Examples: Clustering, feature or dimensionality reduction, etc.

Massachusetts Institute of Technology
MIT Introduction to Deep Learning
introtodeeplearning.com @MITDeepLearning
1/6/26

Generative modeling

Goal: Take as input training samples from some distribution and learn a model that represents that distribution

Density Estimation

Sample Generation

Input samples

Training data $\sim P_{data}(x)$

Generated samples

Generated $\sim P_{model}(x)$

How can we learn $P_{model}(x)$ similar to $P_{data}(x)$?

Massachusetts Institute of Technology
MIT Introduction to Deep Learning
introtodeeplearning.com @MITDeepLearning
1/6/26

Density Estimation:
Given some samples that follow a distribution, learn a model that can learn the probability distribution from which the samples are generated

Sample Generation:
If probability distribution is learned, we can sample from this distribution to generate brand new instances that did not exist in the training data.

Why generative models? Debiasing

Capable of uncovering **underlying features** in a dataset

Homogeneous skin color, pose

VS

Diverse skin color, pose, illumination

How can we use this information to create fair and representative datasets?

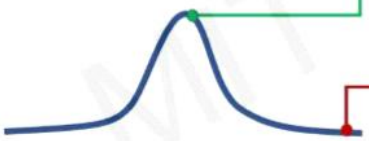
"De-biasing"

Software Lab!

Why generative models? Outlier detection

- **Problem:** How can we detect when we encounter something new or rare?
- **Strategy:** Leverage generative models, detect outliers in the distribution
- Use outliers during training to improve even more!

95% of Driving Data:
(1) sunny, (2) highway, (3) straight road



Detect outliers to avoid unpredictable behavior when training

Edge Cases

Harsh Weather

Pedestrians

Why generative models? Sample generation

- **Generative models learn probability distributions**
- **Sampling** from that distribution → new data instances
- **Backbone of Generative AI:** generate language, images, and more

Natural Language

6.S191 Guest Lectures!

Images & Videos

Molecules

6.S191 Guest Lecture!

Latent variable models

Autoencoders and Variational Autoencoders (VAEs)

Generative Adversarial Networks (GANs)

What is a latent variable?

Myth of the Cave

From Plato's Republic

What is a latent variable?

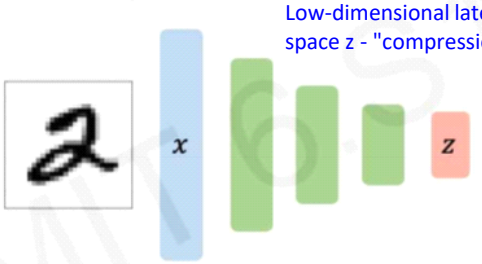
Can we learn the **true explanatory factors**, e.g. latent variables, from only observed data?

Autoencoders

Autoencoders: background

Unsupervised approach for learning a **lower-dimensional** feature representation from unlabeled training data

Low-dimensional latent-space z - "compression"

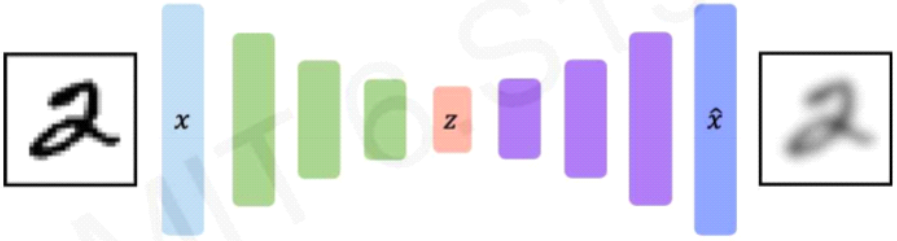


Why do we care about a low-dimensional z?

"Encoder" learns mapping from the data, x, to a low-dimensional latent space, z

Autoencoders: background

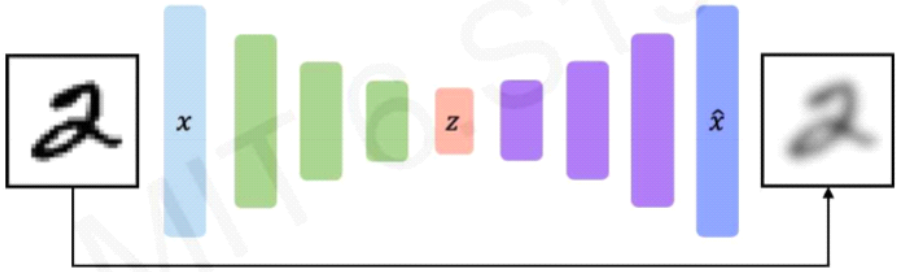
How can we learn this latent space?
Train the model to use these features to **reconstruct the original data**



"Decoder" learns mapping back from latent space, z, to a reconstructed observation, $\hat{x}$

Autoencoders: background

How can we learn this latent space?
Train the model to use these features to **reconstruct the original data**

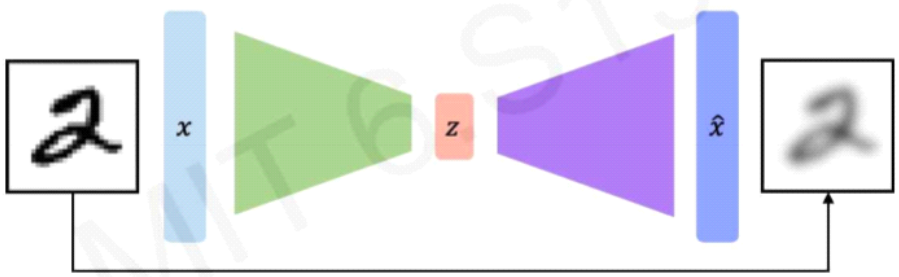


$$\mathcal{L}(x, \hat{x}) = \|x - \hat{x}\|^2$$

Loss function doesn't use any labels!!

Autoencoders: background

How can we learn this latent space?
Train the model to use these features to **reconstruct the original data**



$$\mathcal{L}(x, \hat{x}) = \|x - \hat{x}\|^2$$

Loss function doesn't use any labels!!

Dimensionality of latent space → reconstruction quality

Autoencoding is a form of compression!
Smaller latent space will force a larger training bottleneck

2D latent space | 5D latent space | Ground Truth

Autoencoders for representation learning

Bottleneck hidden layer forces network to learn a compressed latent representation

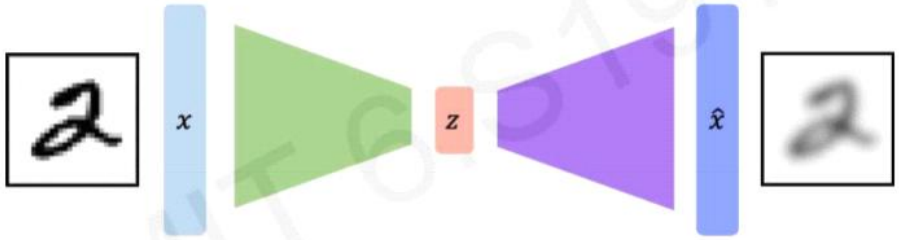
Reconstruction loss forces the latent representation to capture (or encode) as much "information" about the data as possible

Autoencoding = **Auto**matically **encoding** data; "Auto" = **self**-encoding

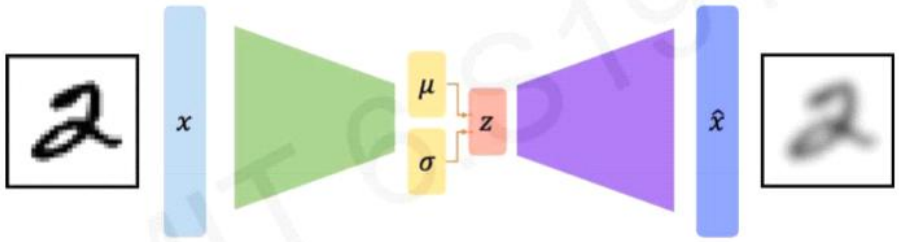
Variational Autoencoders (VAEs)

Traditional autoencoders

Not suitable for sample generation!



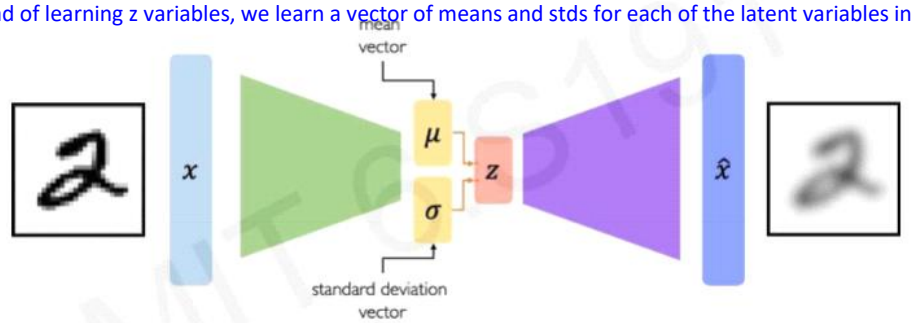
VAEs: key difference with traditional autoencoder



Mean and std can help us parameterize the probability distribution that can enable sampling to produce new data instances

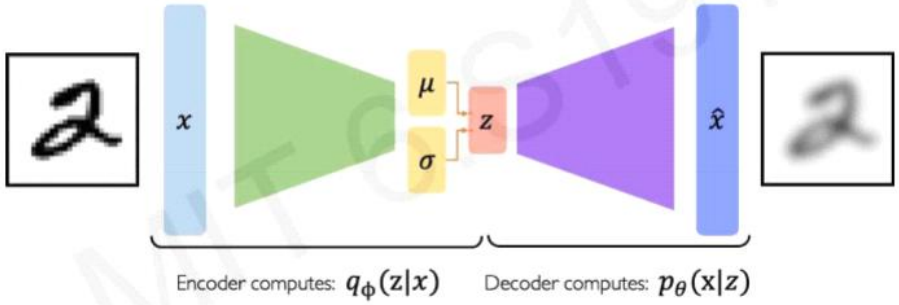
VAEs: key difference with traditional autoencoder

Instead of learning z variables, we learn a vector of means and stds for each of the latent variables in z

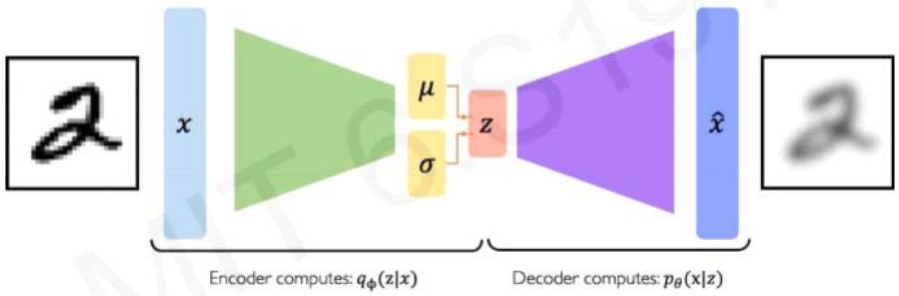


Variational autoencoders are a probabilistic twist on autoencoders!
Sample from the mean and standard deviation to compute latent sample

VAE optimization

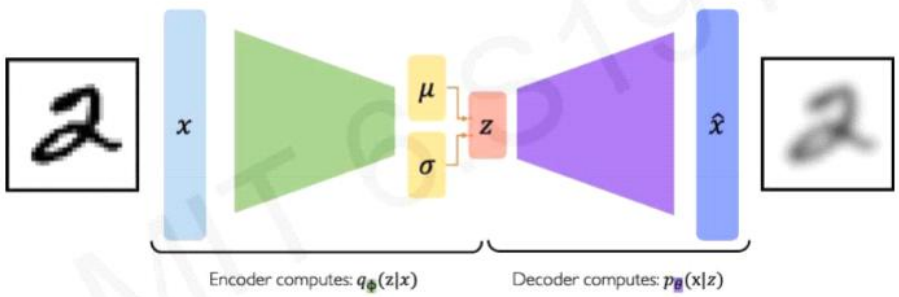


VAE optimization



$\mathcal{L}(\phi, \theta, x)$ = (reconstruction loss) + (regularization term)

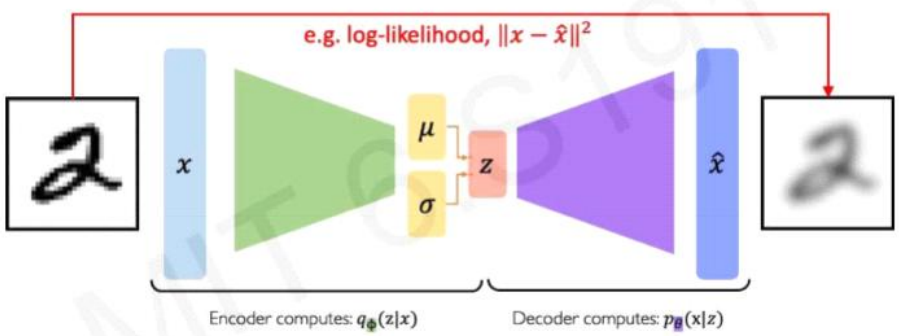
VAE optimization



$\mathcal{L}(\phi, \theta, x)$ = (reconstruction loss) + (regularization term)

Distributional loss - helps us learn good probability distributions

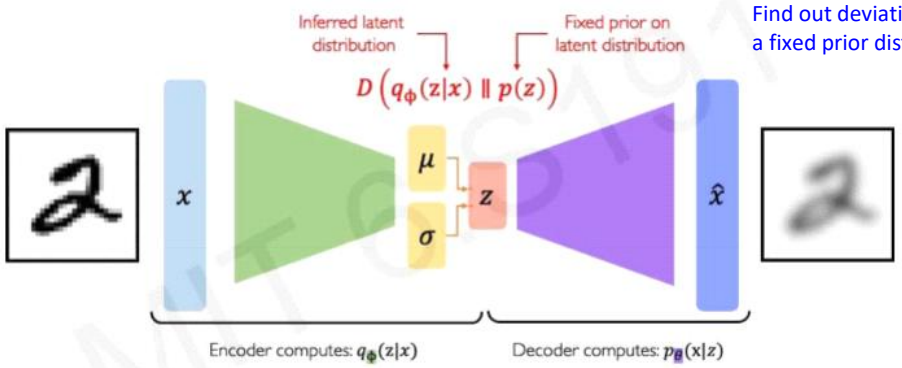
VAE optimization



$\mathcal{L}(\phi, \theta, x)$ = (reconstruction loss) + (regularization term)

VAE optimization

Find out deviation of distribution from a fixed prior distribution



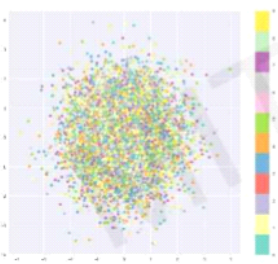
$$\mathcal{L}(\phi, \theta, x) = \text{(reconstruction loss)} + \boxed{\text{(regularization term)}}$$

Priors on the latent distribution

$$D\left(q_\phi(z|x) \parallel p(z)\right)$$

Inferred latent distribution

Fixed prior on latent distribution



Common choice of prior – Normal Gaussian:

$$p(z) = \mathcal{N}(\mu = 0, \sigma^2 = 1)$$

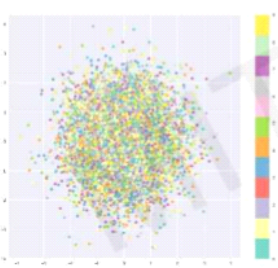
- Encourages encodings to distribute encodings evenly around the center of the latent space
- Penalize the network when it tries to "cheat" by clustering points in specific regions (i.e., by memorizing the data)

Priors on the latent distribution

$$D\left(q_\phi(z|x) \parallel p(z)\right)$$

$$= -\frac{1}{2}\sum_{j=0}^{k-1}\left(\sigma_j + \mu_j^2 - 1 - \log \sigma_j\right)$$

KL-divergence between the two distributions



Common choice of prior – Normal Gaussian:

$$p(z) = \mathcal{N}(\mu = 0, \sigma^2 = 1)$$

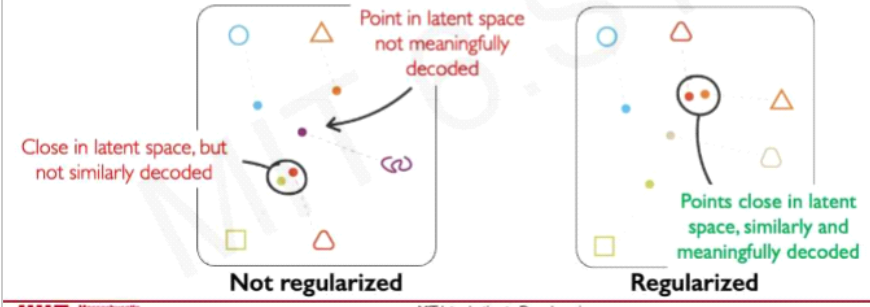
- Encourages encodings to distribute encodings evenly around the center of the latent space
- Penalize the network when it tries to "cheat" by clustering points in specific regions (i.e., by memorizing the data)

Intuition on regularization and the Normal prior

What are the qualities of a good latent space?

What properties do we want to achieve from regularization?

1. **Continuity:** points that are close in latent space → similar content after decoding
2. **Completeness:** sampling from latent space → "meaningful" content after decoding



Intuition on regularization and the Normal prior

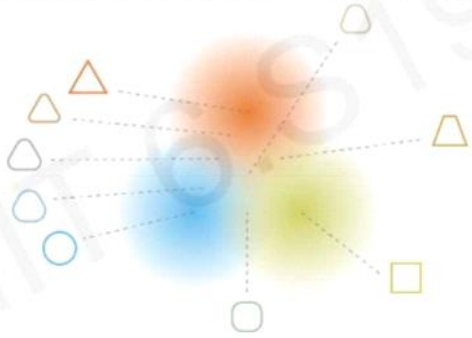
1. **Continuity**: points that are close in latent space → similar content after decoding
2. **Completeness:** sampling from latent space → "meaningful" content after decoding

Encoding as a distribution does not guarantee these properties!

Normal prior → continuity + completeness

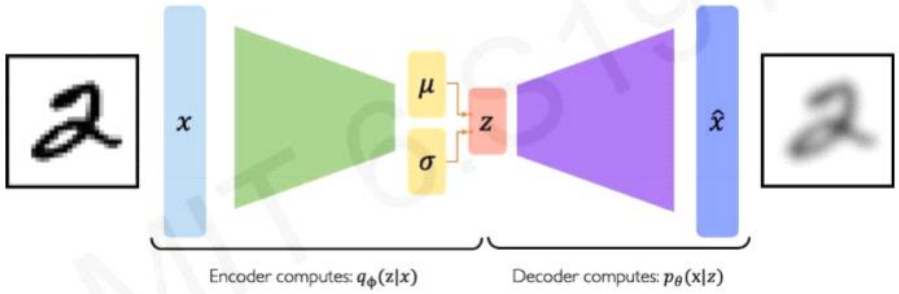
Intuition on regularization and the Normal prior

1. **Continuity**: points that are close in latent space → similar content after decoding
2. **Completeness:** sampling from latent space → "meaningful" content after decoding



Regularization with Normal prior helps enforce **continuity & completeness** in the latent space.

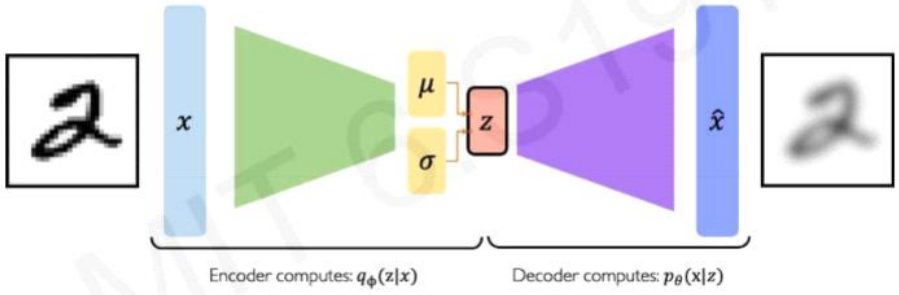
VAE computation graph



$\mathcal{L}(\phi, \theta, x)$ = (reconstruction loss) + (regularization term)

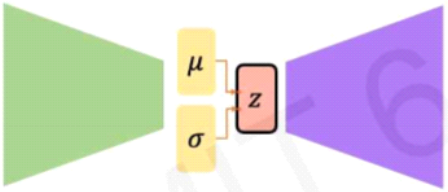
VAE computation graph

Problem: We cannot backpropagate gradients through sampling layers!



$\mathcal{L}(\phi, \theta, x)$ = (reconstruction loss) + (regularization term)

Reparametrizing the sampling layer



Key Idea:

~~$z \sim \mathcal{N}(\mu, \sigma^2)$~~

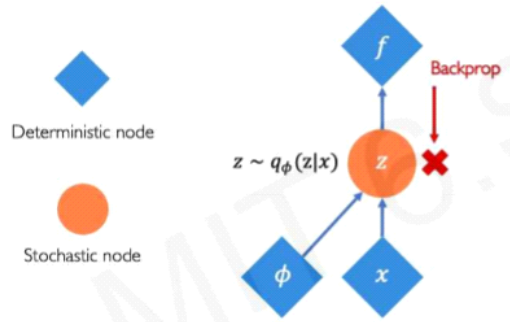
Consider the sampled latent vector z as a sum of

- a fixed μ vector,
- and fixed σ vector, scaled by random constants drawn from the prior distribution

$$\Rightarrow z = \mu + \sigma \odot \varepsilon$$

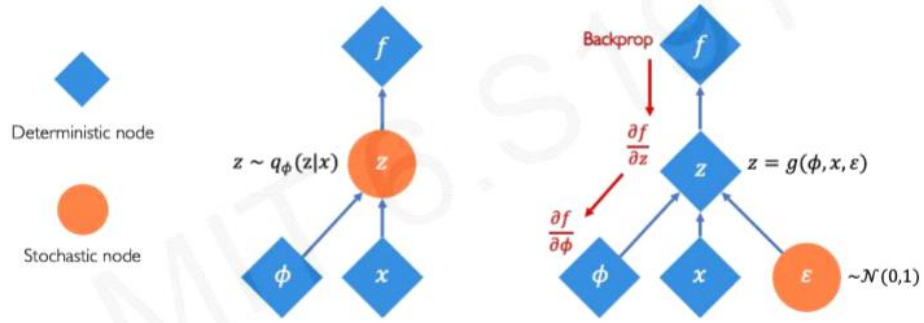
where $\varepsilon \sim \mathcal{N}(0,1)$

Reparametrizing the sampling layer



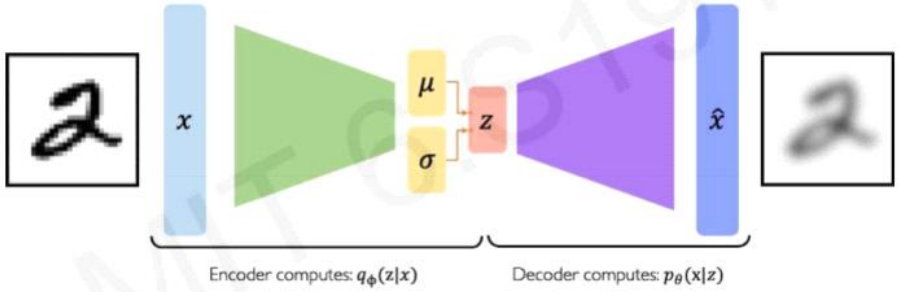
Original form

Reparametrizing the sampling layer



Original form Reparametrized form

VAE computation graph



Sample from latent space and decode to generate new samples

VAEs: Latent perturbation

Slowly increase or decrease a **single latent variable**
Keep all other variables fixed

Head pose

Different dimensions of z encodes **different interpretable latent features**

VAEs: Latent perturbation

Smile

Head pose

Ideally, we want latent variables that are uncorrelated with each other

Enforce diagonal prior on the latent variables to encourage independence

Disentanglement

Latent space disentanglement with β-VAEs

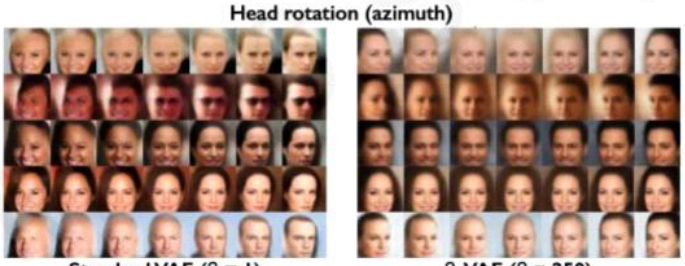
β-VAE loss:

$$\mathcal{L}(\theta, \phi; \mathbf{x}, \mathbf{z}, \beta) = \underbrace{\mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}[\log p_\theta(\mathbf{x}|\mathbf{z})]}_{\text{Reconstruction term}} - \underbrace{\beta D_{KL}(q_\phi(\mathbf{z}|\mathbf{x}) \parallel p(\mathbf{z}))}_{\text{Regularization term}}$$

β > 1: constrain latent bottleneck, encourage efficient latent encoding → disentanglement

Head rotation (azimuth)

Smile also changing!



Smile relatively constant!

Standard VAE (β = 1) **β-VAE (β = 250)**

Why latent variable models? Debiasing

Capable of uncovering **underlying latent variables** in a dataset

Homogeneous skin color, pose

VS

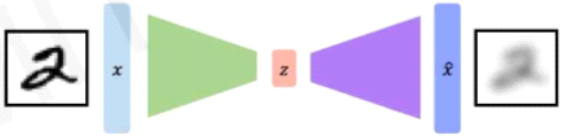
Diverse skin color, pose, illumination

How can we use latent distributions to create fair and representative datasets?

Software Lab!

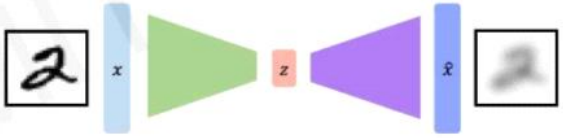
VAE summary

1. Compress representation of world to something we can use to learn



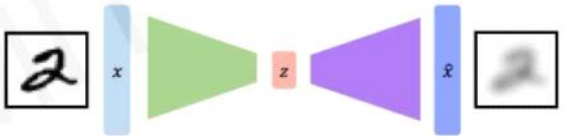
VAE summary

1. Compress representation of world to something we can use to learn
2. Reconstruction allows for unsupervised learning (no labels!)



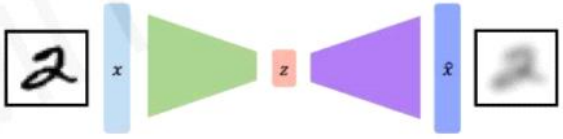
VAE summary

1. Compress representation of world to something we can use to learn
2. Reconstruction allows for unsupervised learning (no labels!)
3. Reparameterization trick to train end-to-end



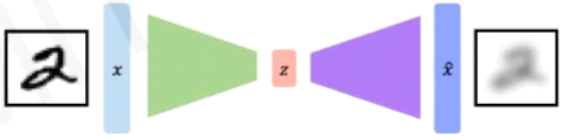
VAE summary

1. Compress representation of world to something we can use to learn
2. Reconstruction allows for unsupervised learning (no labels!)
3. Reparameterization trick to train end-to-end
4. Interpret hidden latent variables using perturbation



VAE summary

1. Compress representation of world to something we can use to learn
2. Reconstruction allows for unsupervised learning (no labels!)
3. Reparameterization trick to train end-to-end
4. Interpret hidden latent variables using perturbation
5. Generating new examples



Generative Adversarial Networks (GANs)

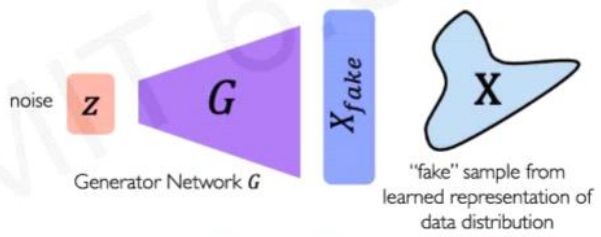
What if we just want to sample?

Only care about sample generation

Idea: don't explicitly model density, and instead just sample to generate new instances.

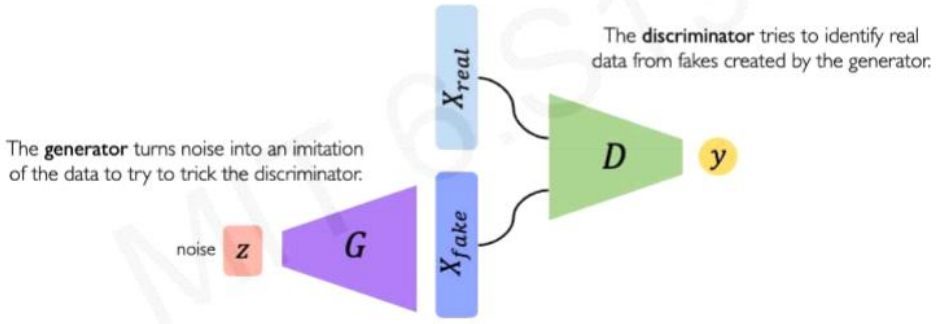
Problem: want to sample from complex distribution – can't do this directly!

Solution: sample from something simple (e.g., noise), learn a transformation to the data distribution.



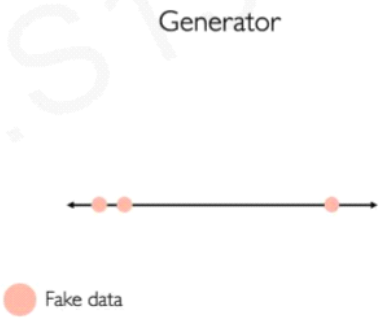
Generative Adversarial Networks (GANs)

Generative Adversarial Networks (GANs) are a way to make a generative model by having two neural networks compete with each other.



Intuition behind GANs

Generator starts from noise to try to create an imitation of the data.



Intuition behind GANs

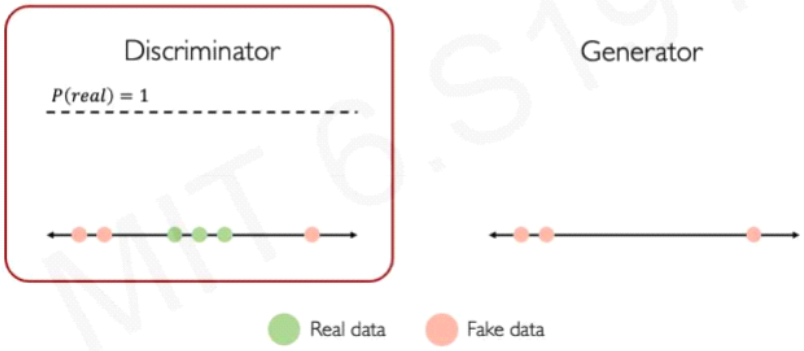
Discriminator looks at both real data and fake data created by the generator.

Intuition behind GANs

Discriminator looks at both real data and fake data created by the generator.

Intuition behind GANs

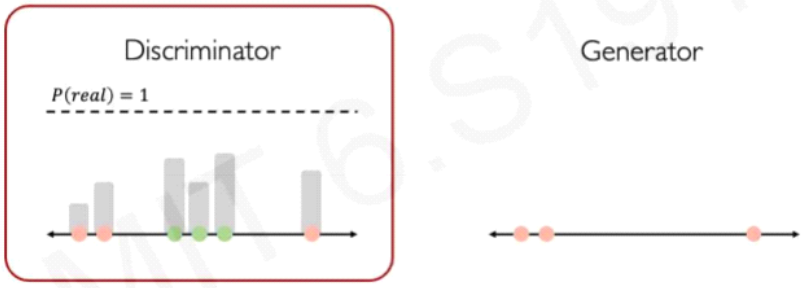
Discriminator tries to predict what's real and what's fake.



Real data Fake data

Intuition behind GANs

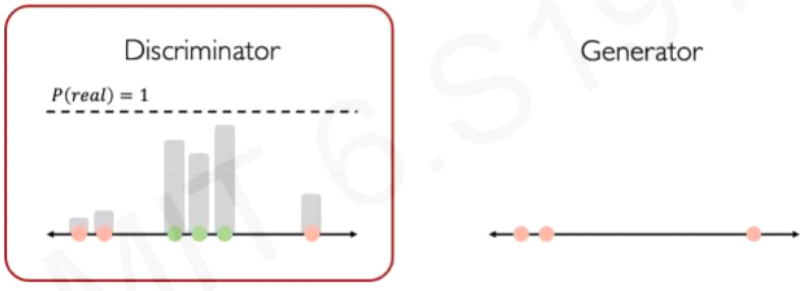
Discriminator tries to predict what's real and what's fake.



Real data Fake data

Intuition behind GANs

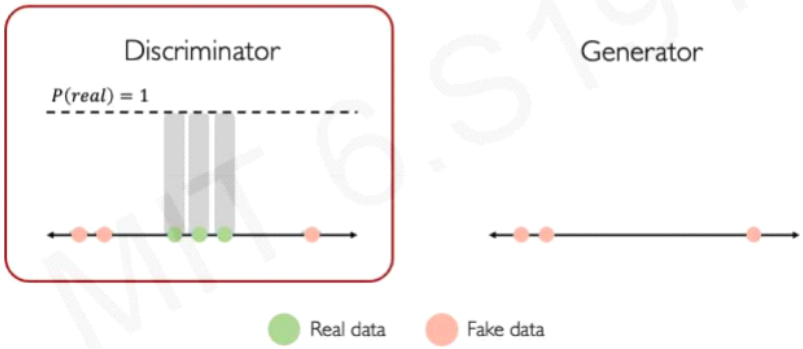
Discriminator tries to predict what's real and what's fake.



Real data Fake data

Intuition behind GANs

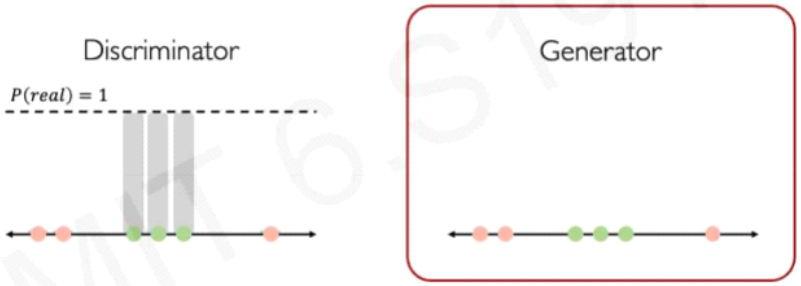
Discriminator tries to predict what's real and what's fake.



Real data Fake data

Intuition behind GANs

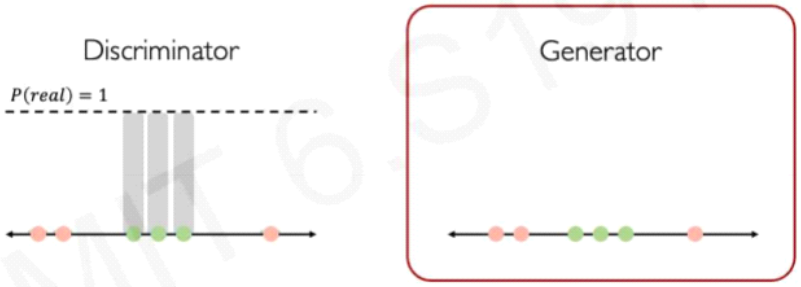
Generator tries to improve its imitation of the data.



Real data Fake data

Intuition behind GANs

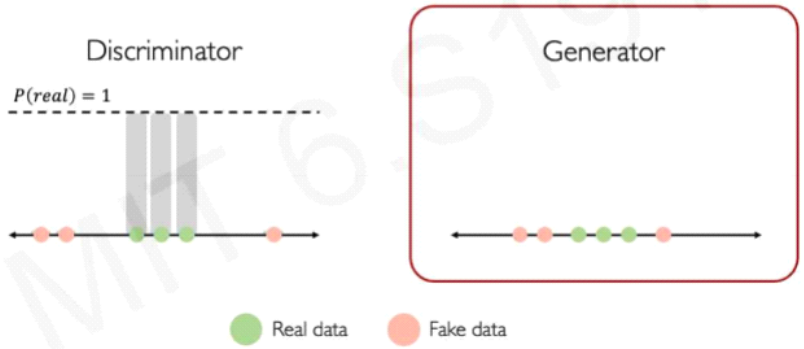
Generator tries to improve its imitation of the data.



Real data Fake data

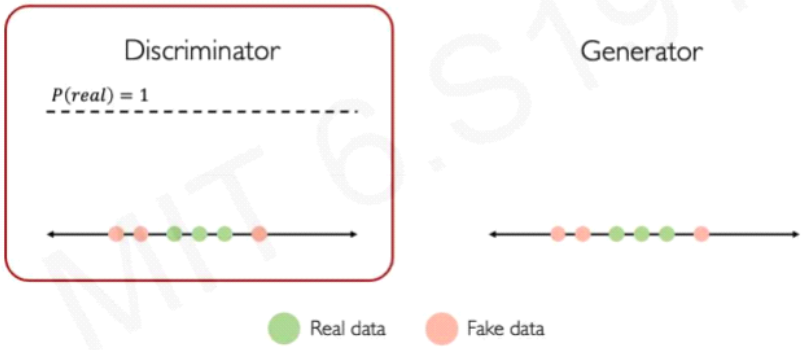
Intuition behind GANs

Generator tries to improve its imitation of the data.



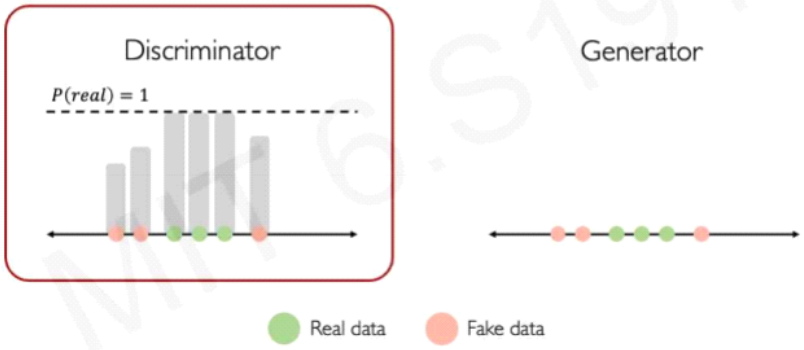
Intuition behind GANs

Discriminator tries to predict what's real and what's fake.



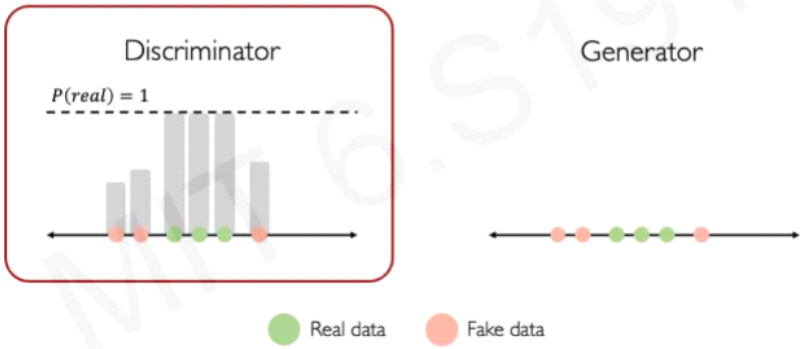
Intuition behind GANs

Discriminator tries to predict what's real and what's fake.



Intuition behind GANs

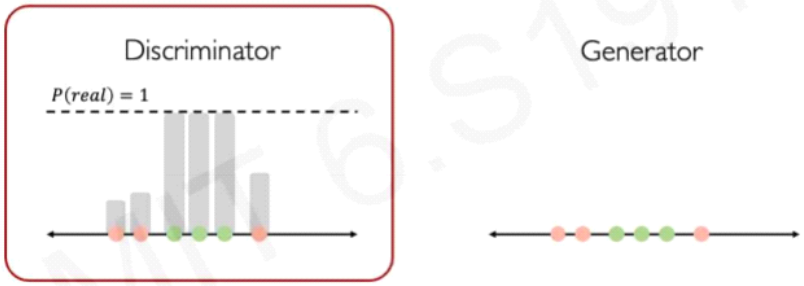
Discriminator tries to predict what's real and what's fake.



Real data Fake data

Intuition behind GANs

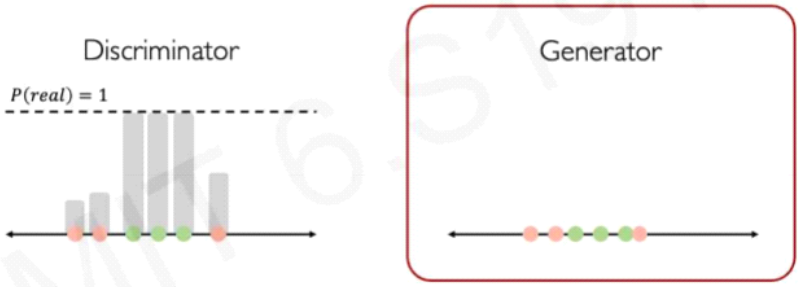
Discriminator tries to predict what's real and what's fake.



Real data Fake data

Intuition behind GANs

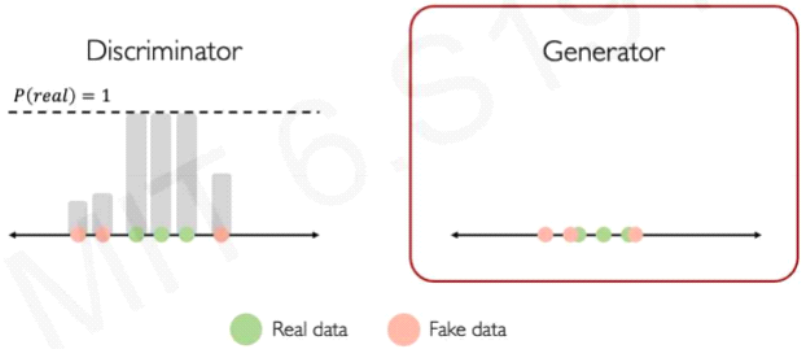
Generator tries to improve its imitation of the data.



Real data Fake data

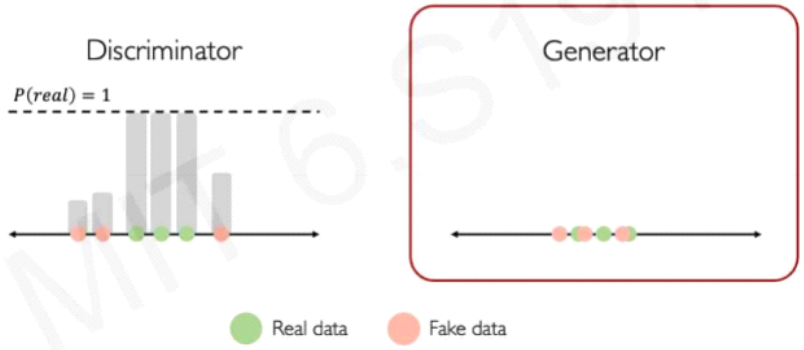
Intuition behind GANs

Generator tries to improve its imitation of the data.



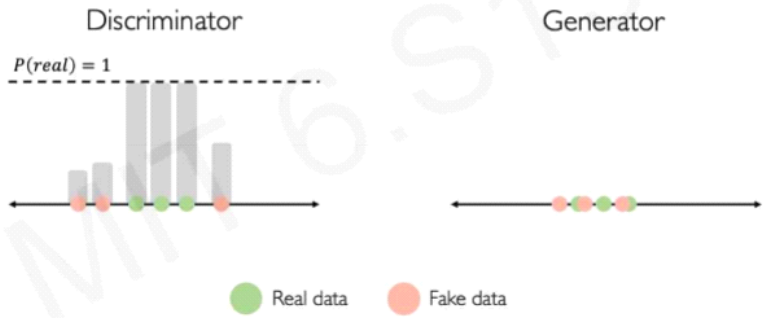
Intuition behind GANs

Generator tries to improve its imitation of the data.

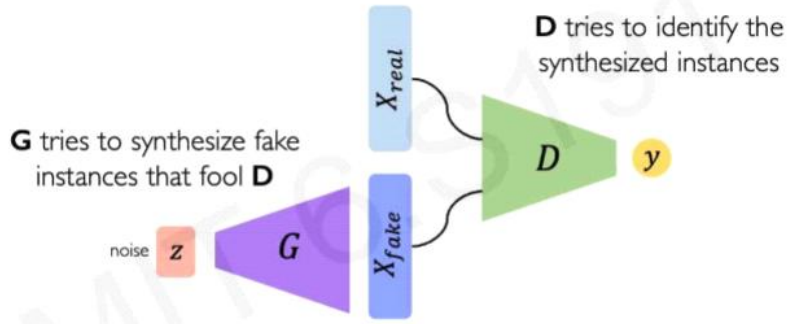


Intuition behind GANs

Discriminator tries to identify real data from fakes created by the generator.
Generator tries to create imitations of data to trick the discriminator.

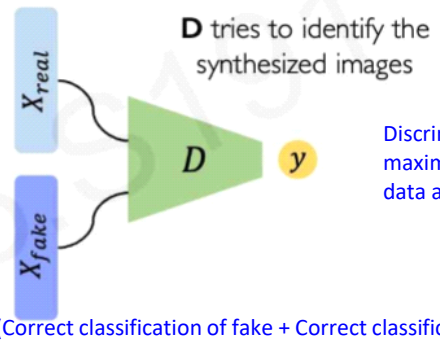


Training GANs



Training: adversarial objectives for **D** and **G**
Global optimum: G reproduces the true data distribution

Training GANs: loss function



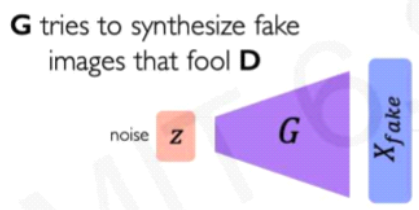
Discriminator's Goal is to maximize the probability of real data as real and fake data as fake.

(Correct classification of fake + Correct classification of Real)

$$\arg\max_{D} \mathbb{E}_{\mathbf{z},\mathbf{x}}\left[\underbrace{\log D(G(\mathbf{z}))}_{\text{Fake}} + \underbrace{\log\left(1 - D(\mathbf{x})\right)}_{\text{Real}}\right]$$

Wrt to Discriminator

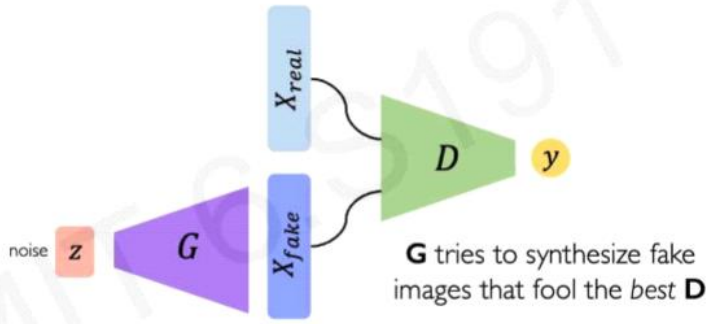
Training GANs: loss function



Generator goal is to minimize the ability of Discriminator in classifying fake as fake

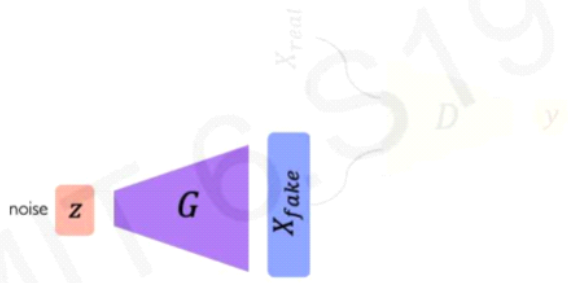
$$\arg\min_{G} \mathbb{E}_{\mathbf{z},\mathbf{x}}\left[\log D(G(\mathbf{z})) + \log\left(1 - D(\mathbf{x})\right)\right]$$

Training GANs: loss function



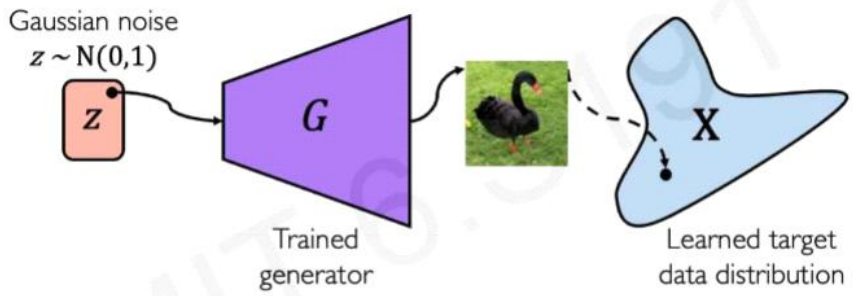
$$\arg\min_{G}\max_{D} \mathbb{E}_{\mathbf{z},\mathbf{x}}[\ \log D(G(\mathbf{z})) + \log(1 - D(\mathbf{x}))\ ]$$

Generating new data with GANs

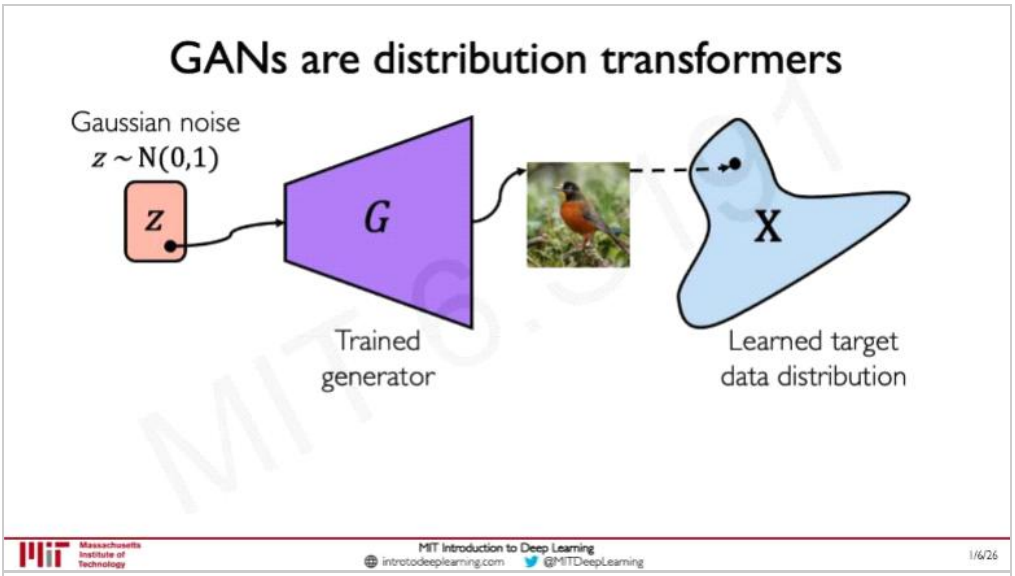


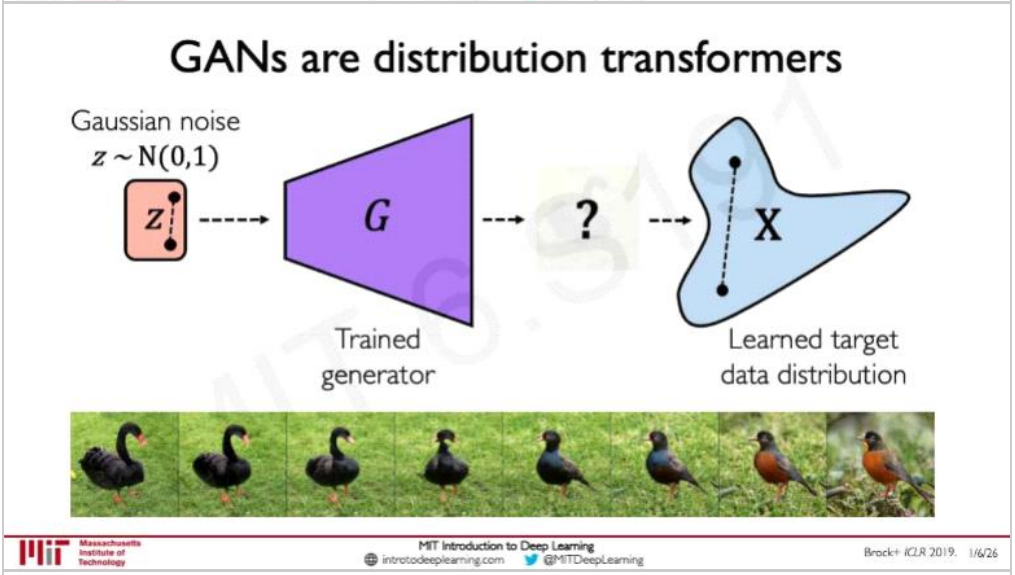
After training, use generator network to create **new data** that's never been seen before.

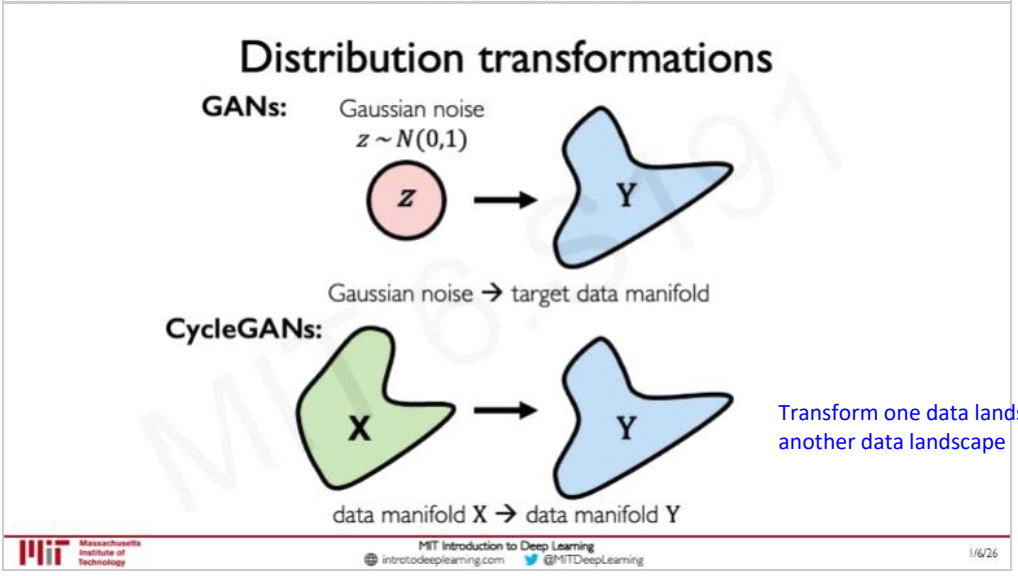
GANs are distribution transformers



Distribution transformation is the concept of going from Gaussian Noise to a new learned distribution.



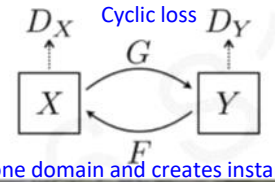




Transform one data landscape to another data landscape

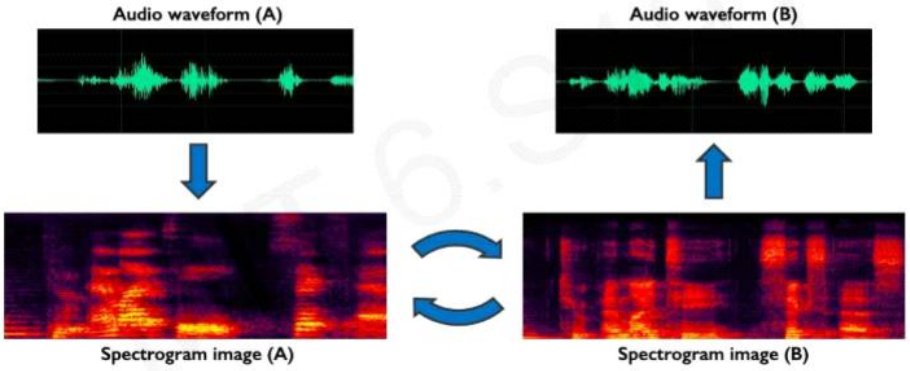
CycleGAN: domain transformation

CycleGAN learns transformations across domains with unpaired data.



Takes instance from one domain and creates instances in another domain

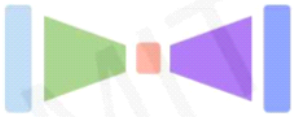
CycleGAN: transforming speech



Deep Generative Modeling: Summary

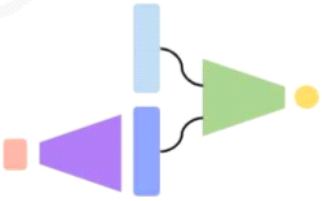
Autoencoders and Variational Autoencoders (VAEs)

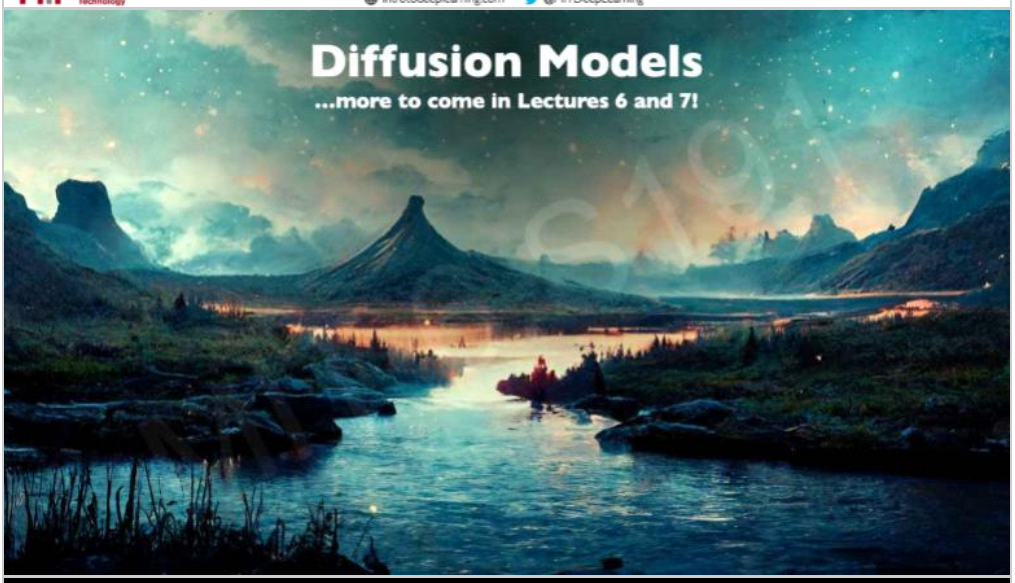
Learn lower-dimensional **latent space** and **sample** to generate input reconstructions



Generative Adversarial Networks (GANs)

Competing **generator** and **discriminator** networks



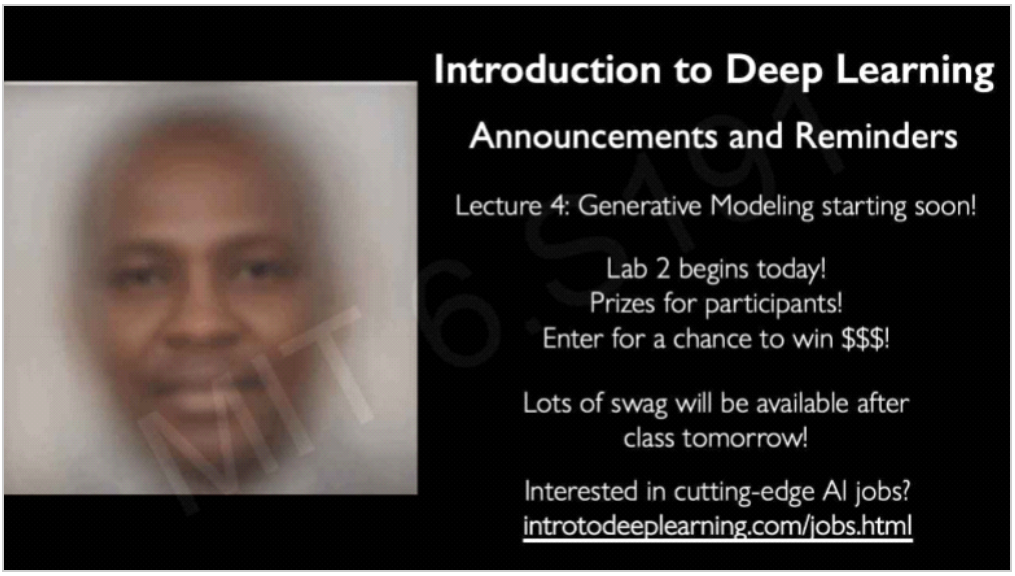


MIT
Introduction to Deep Learning

Lab 2: Facial Detection Systems

Link to download labs:
introtodeeplearning.com#schedule
github.com/MITDeepLearning/introtodeeplearning

1. Open the lab in Google Colab
2. Start executing code blocks and filling in the #TODOs
3. Need help? Come to 32-123!


Introduction to Deep Learning
Announcements and Reminders
Lecture 4: Generative Modeling starting soon!
Lab 2 begins today!
Prizes for participants!
Enter for a chance to win $$$!
Lots of swag will be available after class tomorrow!
Interested in cutting-edge AI jobs?
introtodeeplearning.com/jobs.html
MIT 6.S191